



Inspiring Excellence

CSE370 : Database Systems Project Report

Project Title : Political Promise Tracker

Group No : _01_, CSE370 Lab Section : 02, Spring 2026		
ID	Name	Contribution
24101152	Ibrahim Faruk Bin Abul Kashem	Worked on the completion submission page where users can report fulfilled promises with proof, and built the entire moderator panel including the dashboard, promise and completion review, proof verification, politician edit approvals, and user ban management.
24101156	Hamim Farhat Dipu	Worked on all frontend and backend components related to user interaction with promises, including promise submission with proof upload, the promise detail page with comments and AJAX ratings,

		search functionality, and the user profile page with avatar upload.
24101157	Mohammad Nahiyan Rahman	Contributed to the development of both the backend and frontend of the PolitiTrack platform. Implemented secure authentication, database functionality, and dynamic politician-related features using PHP and SQL. Designed responsive UI components, homepage layouts, and politician detail pages with ratings and promise tracking features.

Table of Contents

Section No	Content	Page No
1	Introduction	03
2	Project Features	04
3	ER/EER Diagram	05
4	Schema Diagram	06
5	Normalization	07
6	Frontend Development	08
7	Backend Development	16
8	Source Code Repository	23
9	Conclusion	24
10	References	25

Introduction

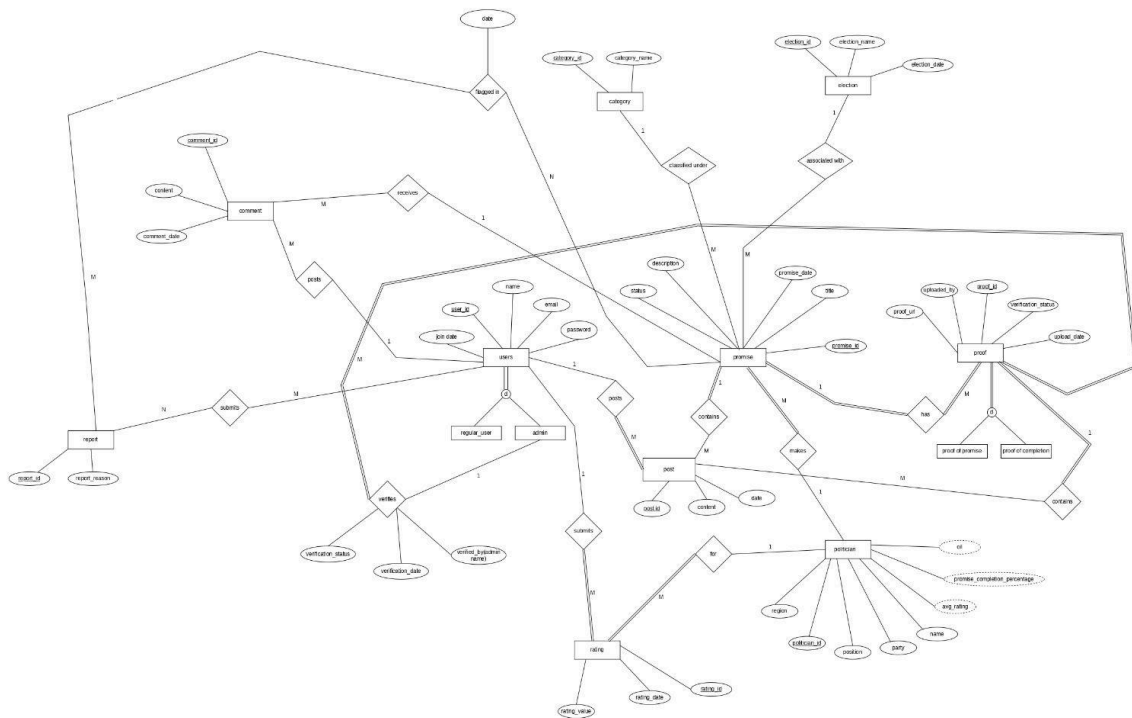
PolitiTrack is a community-driven, web-based political accountability platform built using PHP and MySQL. The core idea is to bridge the gap between politicians and the public by creating a structured space where citizens can document, track, and evaluate promises made by political figures. Any registered user can submit a promise that a politician has made, attach supporting proof (such as images or documents), and link it to a relevant category. Other users can then comment on these posts, rate the politician's overall performance, and submit completion reports when a promise has been fulfilled. A dedicated moderator panel ensures platform integrity by reviewing all submitted content before it goes live, handling user reports, and managing politician profile edits. The platform promotes transparency, civic engagement, and data-driven political evaluation.

Project Features

ID, Name	Features [3 per member]	
24101157, Mohammad Nahiyan Rahman	Ft 1	Project Setup & Infrastructure
	Ft 2	Home Feed & Politician Listing
	Ft 3	Politician Detail Page & Ratings
24101156, Hamim Farhat Dipu	Ft 1	Submit Promise with Proof Upload
	Ft 2	Promise Detail Page (Comments & Ratings)
	Ft 3	Search & User Profile
24101152, Ibrahim Faruk Bin Abul Kashem	Ft 1	Submit Completion
	Ft 2	Moderator Dashboard & Promise/Completion Review
	Ft 3	Moderator Reports, Edits & User Management

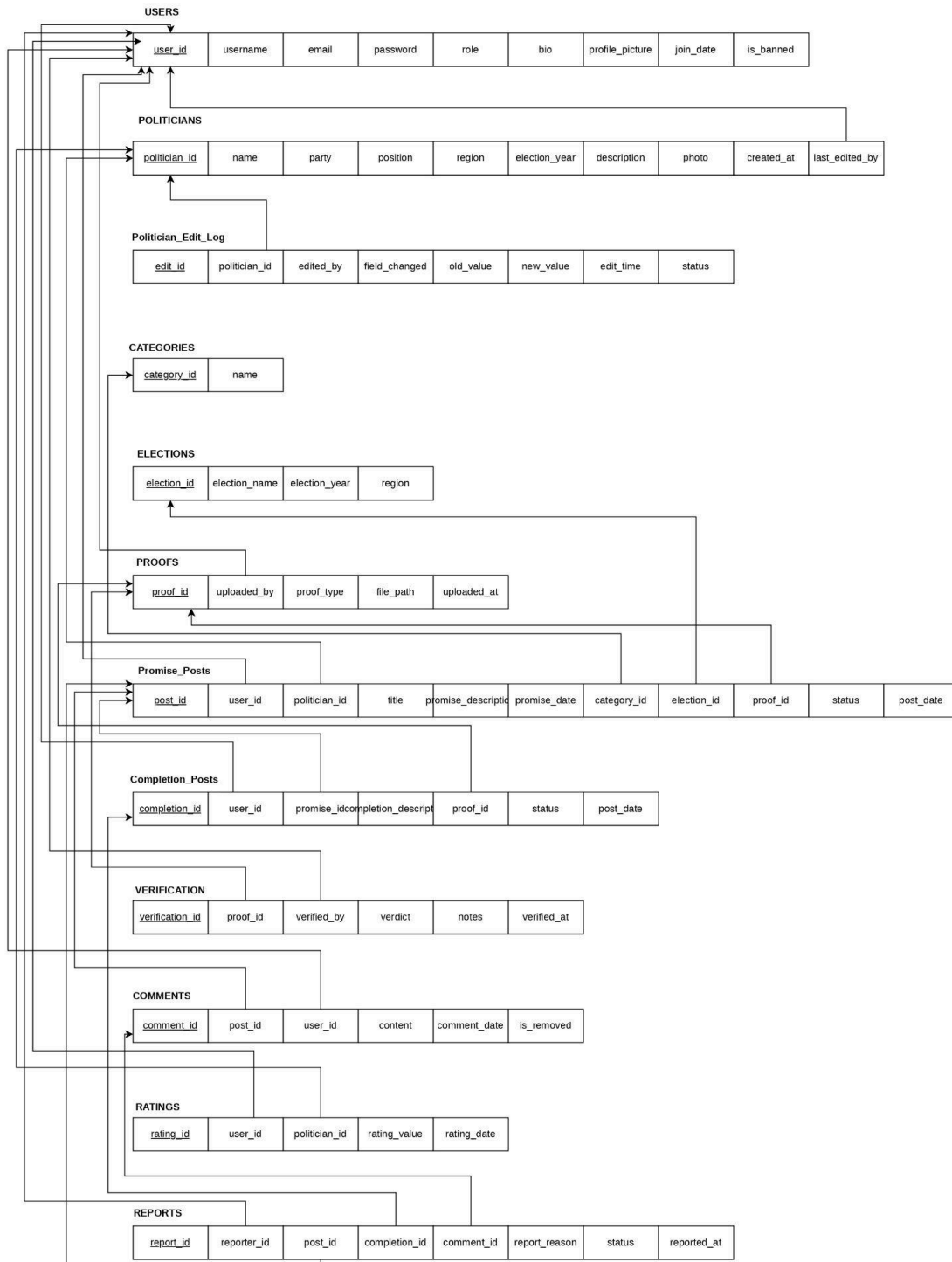
ER/EER Diagram

[EER Diagram Link](#)



Schema Diagram

[Schema Diagram Link](#)



Normalization

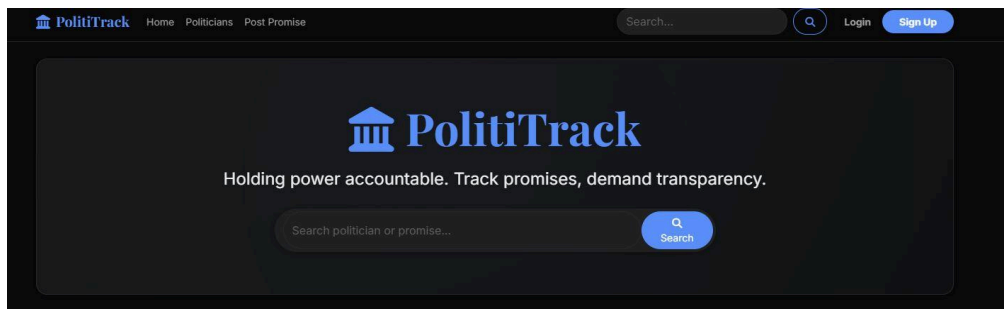
- a. The converted schema is already in 1NF, as there are no multivalued, composite attributes or nested relations.
- b. The converted schema is already in 2NF, as there are no partial dependencies in any Relation.
- c. The converted schema is already in 3NF, as there are no transitive dependencies (no non key attribute depends on another non key attribute).

Frontend Development

Contribution of ID : 24101157, Name : Mohammad Nahiyan Rahman
In Frontend Development:

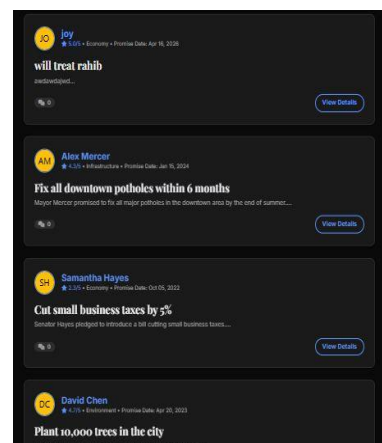
Feature 1: Global UI & Visual Foundation

1. Established the entire visual foundation of PolitiTrack, including a global navigation bar with dynamic login/logout state rendering
2. Created reusable shared header and footer partials used consistently across all pages
3. Developed the main stylesheet with custom Bootstrap overrides for the platform's navy and white color scheme

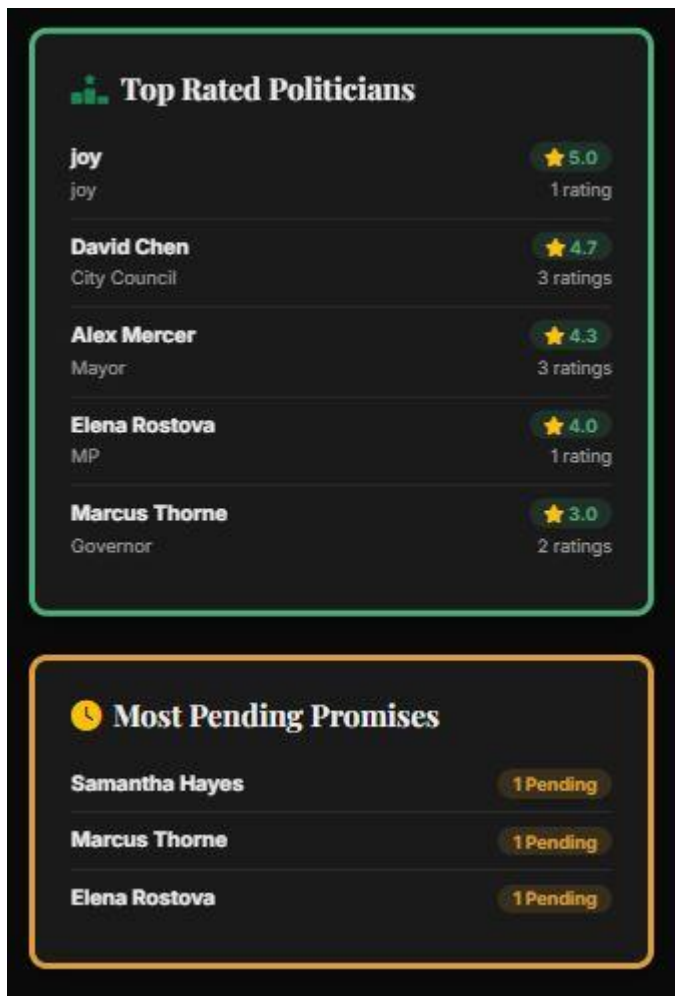


Feature 2: Home Feed & Politician Listing

1. Built the home feed page (index.php) displaying the ten most recently approved promise posts in a card layout
2. Added a sidebar section showing the top five highest-rated politicians and politicians with the most pending promises using dynamic database queries
3. Developed the politician listing page (politicians.php) displaying all politicians in a browsable grid with their party affiliation,



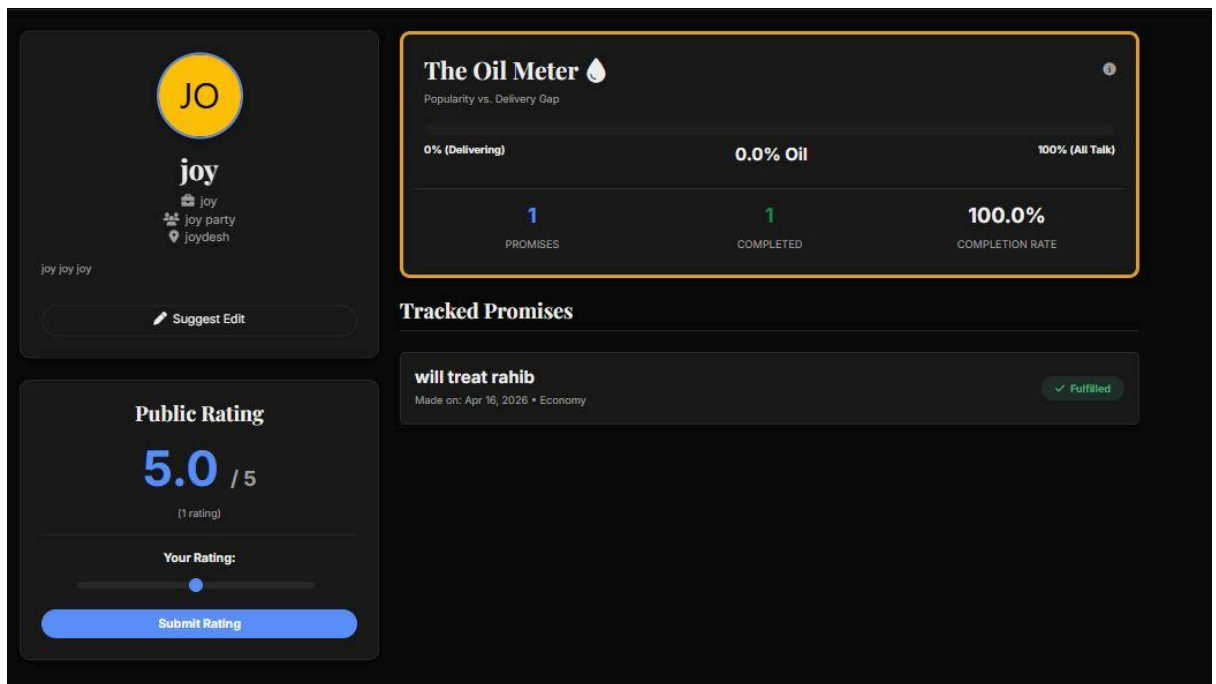
position, and average rating badge



Feature 3: Politician Detail Page (politician.php)

1. Designed a complete politician profile page displaying the politician's photo and biographical information

2. Added a visual star rating widget for user ratings



3. Built a filterable promise history table showing promise statuses including pending, approved, and completed
4. Added a breakdown chart visualizing promise categories

Contribution of ID : 24101156, Name : Hamim Farhat Dipu in Frontend Development :

Feature 4: Submit Promise Page (submit_promise.php)

1. Built a multi-field promise submission form with a live politician name autocomplete input that queries the database as the user types

2. Added a category dropdown, a status selector, and a description text area with a real-time character counter showing remaining characters
3. Added a JavaScript helper file (assets/js/main.js) containing the autocomplete logic, character counter, and drag-and-drop upload functionality

Feature 5: Promise Detail Page (post.php)

1. Built the platform displaying the full promise description and all associated metadata
2. Rendered attached proof inline for images, and as a download link for document files
3. Designed a threaded comment section allowing users to post and view responses beneath each promise
4. Implemented an interactive five-star rating widget that submits ratings via AJAX without requiring a full page reload

Post a New Promise
Hold politicians accountable by tagging a promise they made. Please provide valid proof.

Politician Name * Date Promise Made *

Promise Title *

Promise Description *

Category

☒ **Proof (Required)**
Provide a link to a news article, OR upload a screenshot (Max 20MB).

Proof Link (URL)

OR

Upload File

PoliTrack Home Politicians Post Promise

Alex Mercer
Mayor Mercer promised to fix all major potholes in the downtown area by the end of summer.

☒ **Proof of Promise**
[Screenshot of potholes being fixed]

☒ **Fulfillments**
The potholes were fixed ahead of schedule.
Evidence of Completion: [Screenshot of potholes being fixed]

Discussion (1)
[Comment: I saw the potholes fixed on my way to work.]

Feature 6: Search & User Profile (search.php, profile.php)

Public Rating
4.3 / 5
(3 ratings)

Your Rating:
[Star Rating]

1. Developed the search results page displaying matched politicians and promise posts in separate tabbed sections

PoliTrack Home Politicians Post Promise

Search Results for "Alex Mercer"

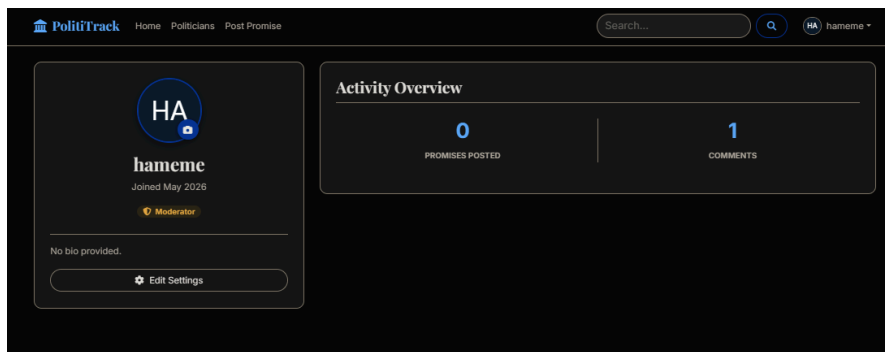
Politicians (5)

- Alex Mercer (Progressive Party)
- Samantha Hayes (Conservative Party)
- David Chen (Green Alliance)
- Marcus Thorne (Independent)
- Elena Rosanova (Progressive Party)

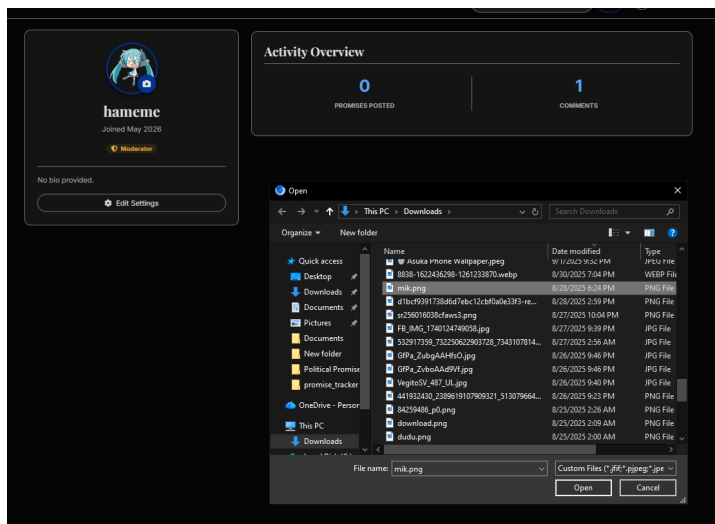
Promises (5)

- Fix all downtown potholes within 6 months (By Alex Mercer)
- Cut small business taxes by 5% (By Samantha Hayes)
- Plant 10,000 trees in the city (By David Chen)
- Publish all government contracts online (By Marcus Thorne)
- Build a new hospital in District 8 (By Elena Rosanova)

2. Built the user profile page showing a user's submitted promises, comments, and ratings in an organized activity history view



3. Added a profile picture update feature using a modal avatar upload interface with a live image preview before submission



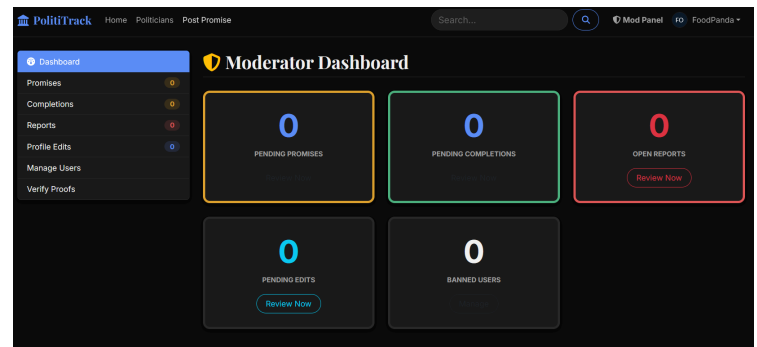
Contribution of ID : 24101152, Name : Ibrahim Faruk Bin Abul Kashem in Frontend Development :

Feature 7: Submit Completion Page **([submit_completion.php](#))**

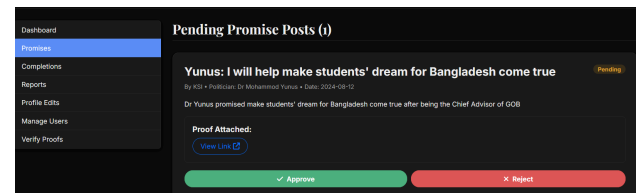
- Built a form with a dropdown listing all approved promises for the user to select from
- Added a description text box and a proof section where users can paste a URL or upload an image/video
- Added client-side validation that shows error messages instantly if required fields are missing before the form is submitted

Feature 8: Moderator Dashboard & Promise/Completion Review

- Designed a consistent sidebar navigation menu used across all moderator pages with live badge counters showing how many pending items are in each section
- Built the moderator dashboard displaying 5 summary stat cards ,pending promises, pending completions, open reports, pending edits, and banned users

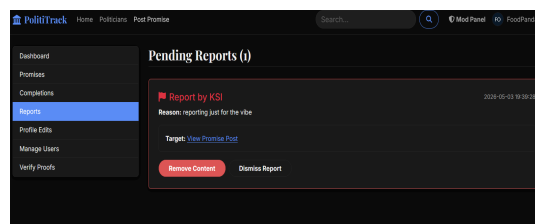


- Built the promise and completion review pages displaying pending submissions in a table with approve/reject buttons and expandable detail rows for viewing full content in line.

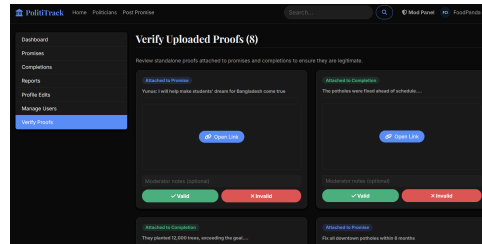


Feature 9: Moderator Reports, Edits & User Management

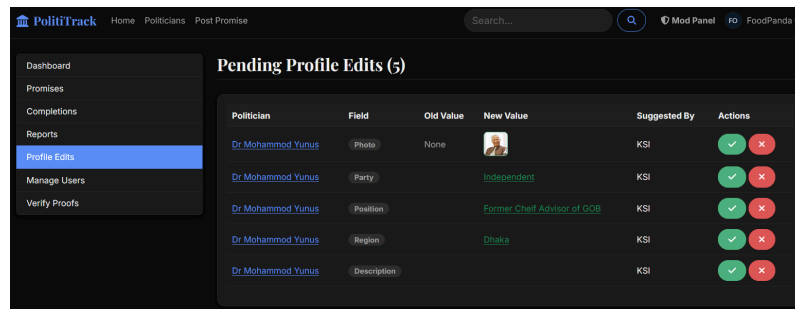
- Built the reports panel showing flagged content alongside the report reason so moderators have full context



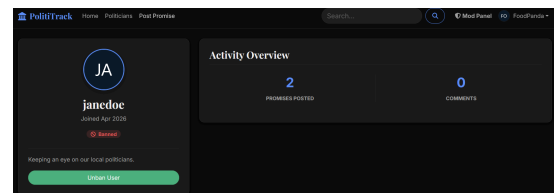
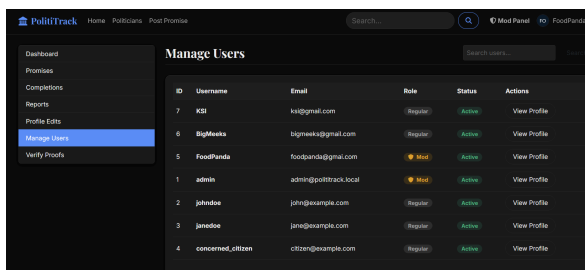
- Built the proof review page displaying all uploaded evidence attached to pending submissions for moderator inspection



- Built the politician edit log page showing old and new values side by side with approve/reject buttons



- Built the user management page listing all users with a ban/unban toggle button next to each one



Backend Development

Contribution of **ID : 24101157, Name : Mohammad Nahiyan Rahman**

In Backend Development:

Feature 1: Project Setup & Infrastructure

1. Configured a secure PDO database connection with error handling
2. Developed the authentication system with session management and access control using helper functions like `requireLogin()` and `requireModerator()`
3. Implemented secure password hashing and verification using PHP's `password_hash()` and `password_verify()` functions
4. Created reusable backend utility functions for CSRF token generation, validation, input sanitization, and rating-related operations

```
require_once '../config/db.php';
require_once '../includes/header.php';
requireModerator();

$search = $_GET['q'] ?? '';
$q = '%' . $search . '%';

$stmt = $pdo->prepare("SELECT * FROM users WHERE username LIKE ? OR email LIKE ? ORDER BY join_date DESC");
$stmt->execute([$q, $q]);
$users = $stmt->fetchAll();
?>

<div class="row">
  <div class="col-md-3 mb-4">
    <div class="list-group shadow-sm">
```

5. Built the backend functionality for login, signup, and logout pages, including credential verification, duplicate account checks, session handling, and redirect logic

Feature 2: Home Feed & Politician Listing

1. Implemented backend functionality for the home feed and politician listing sections using optimized SQL queries
2. Developed queries to retrieve the ten most recently approved promises along with related politician and category information
3. Calculated average politician ratings using SQL subqueries


```

<?php
require_once 'config/db.php';
require_once 'includes/header.php';

// Fetch all politicians with stats
$stmt = $pdo->query("
    SELECT p.*,
           (SELECT AVG(rating_value) FROM ratings WHERE politician_id = p.politician_id) as avg_rating,
           (SELECT COUNT(rating_id) FROM ratings WHERE politician_id = p.politician_id) as rating_count,
           (SELECT COUNT(*) FROM promise_posts WHERE politician_id = p.politician_id AND status = 'approved') as total_promises,
           (SELECT COUNT(*) FROM completion_posts cp JOIN promise_posts pp ON cp.promise_id = pp.post_id WHERE pp.politician_id = p.politician_id AND cp.status
    FROM politicians p
    ORDER BY name ASC
");
$politicians = $stmt->fetchAll();
?>

<div class="row mb-4 align-items-center">

```

4. Added queries for displaying the top five highest-rated politicians and politicians with the most pending promises in the homepage sidebar
5. Built functionality for retrieving politician records and summary information for listing pages

Feature 3: Politician Detail Page & Ratings

1. Implemented backend functionality for the politician detail page and rating system
2. Created queries to retrieve complete politician profile information
3. Developed functionality to aggregate promise statistics grouped by status
4. Added queries to fetch and manage all user ratings associated with each politician
5. Integrated dynamic average rating calculations and organized rating data for display on the politician detail page

Contribution of ID : 24101156, Name : Hamim Farhat Dipu in Backend Development:

Feature 4: Submit Promise Page (submit_promise.php)

1. Handled a multi-step database transaction for promise submission: first inserting the uploaded proof file record into the proofs table with file type and size validation, unique filename generation, and server-side file move, then inserting the promise post record into promise_posts referencing the new proof ID

```

if ($existing) {
    $politician_id = $existing;
} else {
    $stmt = $pdo->prepare("INSERT INTO politicians (name) VALUES (?)");
    $stmt->execute([$politician_name]);
    $politician_id = $pdo->lastInsertId();
    $_SESSION['toast'] = ['type' => 'info', 'message' => 'New politician profile created.'];
}

```

```

$stmt = $pdo->prepare("INSERT INTO promise_posts (user_id, politician_id, title, promise_description, promise_date, category_id, proof_id) VALUES (?, ?, ?, ?, ?, ?, ?)");
$stmt->execute([$SESSION['user_id'], $politician_id, $title, $desc, $date, $cat_id, $proof_id]);

```

2. Handled the Proof Upload (calls `handle_proof_upload()` from `functions.php`)

```

function handle_proof_upload($file, $link, $pdo, $user_id) {
    if (empty($link)) {
        // Handle link
        $stmt = $pdo->prepare("INSERT INTO proofs (uploaded_by, proof_type, file_path) VALUES (?, 'link', ?)");
        $stmt->execute([$user_id, $link]);
        return $pdo->lastInsertId();
    }
}

```

3. Set the post status to pending upon insertion so all new submissions await moderator approval before going live

Feature 5: Promise Detail Page (post.php)

1. Performed a complex joined query across the `promise_posts`, `politicians`, `users`, `categories`, and `proofs` tables to assemble all data required for a single post view

```

// Fetch Promise
$stmt = $pdo->prepare("
    SELECT pp.*, p.name AS politician_name, p.photo AS politician_photo, c.name AS category_name,
           pr.proof_type, pr.file_path as proof_path, u.username as author_name
    FROM promise_posts pp
    JOIN politicians p ON pp.politician_id = p.politician_id
    JOIN users u ON pp.user_id = u.user_id
    LEFT JOIN categories c ON pp.category_id = c.category_id
    JOIN proofs pr ON pp.proof_id = pr.proof_id
    WHERE pp.post_id = ? AND pp.status = 'approved'
");
$stmt->execute([$id]);
$post = $stmt->fetch();

```

2. Handled comment submission via a POST action that inserts into the `comments` table with the logged-in user's ID and a timestamp

```

$stmt = $pdo->prepare("INSERT INTO comments (post_id, user_id, content) VALUES (?, ?, ?)");
$stmt->execute([$id, $_SESSION['user_id'], $content]);
header("Location: post.php?id=$id");

```

3. Implemented rating submission using an `INSERT ... ON DUPLICATE KEY UPDATE` pattern on the `ratings` table to prevent duplicate ratings

per user per politician, with the result returned via AJAX for seamless

```
if ($rating >= 1 && $rating <= 5) {  
    $stmt = $pdo->prepare("  
        INSERT INTO ratings (user_id, politician_id, rating_value)  
        VALUES (?, ?, ?)  
        ON DUPLICATE KEY UPDATE rating_value = VALUES(rating_value)  
    ");  
    $stmt->execute([$SESSION['user_id'], $id, $rating]);  
}
```

frontend updates

Feature 6: Search & User Profile

1. Implemented the search backend (search.php) using parameterized LIKE queries against both the politicians table (matching on name, party, and position) and the promise_posts table (matching on title and description), returning both result sets to the view

```
// Search politicians  
$stmt = $pdo->prepare("SELECT * FROM politicians WHERE name LIKE ? OR party LIKE ? OR position LIKE ?");  
$stmt->execute([$q, $q, $q]);  
$politicians = $stmt->fetchAll();
```

```
$stmt = $pdo->prepare("  
    SELECT pp.*, p.name AS politician_name  
    FROM promise_posts pp  
    JOIN politicians p ON pp.politician_id = p.politician_id  
    WHERE (pp.title LIKE ? OR pp.promise_description LIKE ?) AND pp.status = 'approved'  
");  
$stmt->execute([$q, $q]);  
$promise_posts = $stmt->fetchAll();
```

2. Built the profile page backend (profile.php) to retrieve a user's full record along with their associated promise posts, comments, and ratings for display in the activity history

```
// Stats  
$stmt = $pdo->prepare("SELECT COUNT(*) FROM promise_posts WHERE user_id = ?");  
$stmt->execute([$id]);  
$post_count = $stmt->fetchColumn();  
  
$stmt = $pdo->prepare("SELECT COUNT(*) FROM comments WHERE user_id = ?");  
$stmt->execute([$id]);  
$comment_count = $stmt->fetchColumn();  
?>
```

3. Developed the avatar API endpoint (api/update_avatar.php) to handle AJAX POST requests, validate and save the uploaded image to the avatars directory, update the user record in the database, and return a JSON response with the new avatar path for live preview updates

```
// Update DB  
$stmt = $pdo->prepare("UPDATE users SET profile_picture = ? WHERE user_id = ?");  
if ($stmt->execute([$new_name, $user_id])) {  
    // Update session  
}
```

Contribution of ID : 24101152, Name : Ibrahim Faruk Bin Abul Kashem in Backend Development:

Feature 7: Submit Completion Page ([submit_completion.php](#))

- Validated that the selected promise exists in the database and has an approved status before allowing submission

```
submit_completion.php X
Political Promise Tracker > politrack > submit_completion.php
7 $stmt = $pdo->query("SELECT post_id, title FROM promise_posts WHERE status = 'approved' ORDER BY post_date DESC");
8 $promises = $stmt->fetchAll();
```

- Used a database transaction to insert the proof into the [proofs](#) table and the completion into the [completion_posts](#) table together, if either fails, both are cancelled so no incomplete data is saved

```
submit_completion.php X
Political Promise Tracker > politrack > submit_completion.php
12 if ($SERVER['REQUEST_METHOD'] == 'POST') {
13     try {
14         $pdo->beginTransaction();
15         $promise_id = $_POST['promise_id'];
16         $desc = trim($_POST['completion_description']);
17         //proof handle
18         $proof_link = trim($_POST['proof_link'] ?? '');
19         $proof_file = $_FILES['proof_file'] ?? null;
20         if (empty($proof_link) && empty($proof_file['name'])) {
21             throw new Exception("You must provide either a proof link or upload a file.");
22         }
23         $proof_id = handle_proof_upload($proof_file, $proof_link, $pdo, $_SESSION['user_id']);
24         //completion post
25         $stmt = $pdo->prepare("INSERT INTO completion_posts (user_id, promise_id, completion_description, proof_id, status) VALUES
26         $stmt->execute([$_SESSION['user_id'], $promise_id, $desc, $proof_id]);
27         $pdo->commit();
```

- Set the completion's default status to pending so it waits for moderator approval before going live

```
submit_completion.php X
Political Promise Tracker > politrack > submit_completion.php
28 $stmt = $pdo->prepare("INSERT INTO completion_posts (user_id, promise_id, completion_description, proof_id, status) VALUES
29 $stmt->execute([$_SESSION['user_id'], $promise_id, $desc, $proof_id]);
30 $pdo->commit();
```

Feature 8: Moderator Dashboard & Promise/Completion Review

- Protected every moderator page using [requireModerator\(\)](#) which checks the user's session role and redirects non-moderators away immediately

```
1 <?php
2 require_once '../config/db.php';
3 require_once '../includes/header.php';
4 requireModerator();
5
```

```
index.php X
Political Promise Tracker > politrack > moderator > index.php
7 $pending_posts = $pdo->query("SELECT COUNT(*) FROM promise_posts WHERE status = 'pending'")->fetchColumn();
8 $pending_completions = $pdo->query("SELECT COUNT(*) FROM completion_posts WHERE status = 'pending'")->fetchColumn();
9 $open_reports = $pdo->query("SELECT COUNT(*) FROM reports WHERE status = 'pending'")->fetchColumn();
10 $pending_edits = $pdo->query("SELECT COUNT(*) FROM politician_edit_log WHERE status = 'pending'")->fetchColumn();
11 $banned_users = $pdo->query("SELECT COUNT(*) FROM users WHERE is_banned = 1")->fetchColumn();
12 >>
```

- Wrote 5 COUNT queries on the dashboard to fetch the number of pending items from each relevant table
- Wrote the approve/reject handler for promises, verifies the CSRF token then runs an UPDATE query changing the promise status to approved or rejected
- Wrote the approve/reject handler for completions, same as promises but approval also triggers a second UPDATE that marks the parent promise as completed, both wrapped in a transaction

```
posts.php X
Political Promise Tracker > politrack > moderator > posts.php
6 if ($SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action'])) {
7     verifyCsrfToken($_POST['csrf_token']);
8     $post_id = $_POST['post_id'];
9     $action = $_POST['action']; // 'approved' or 'rejected'
10
11     if (in_array($action, ['approved', 'rejected'])) {
12         $stmt = $pdo->prepare("UPDATE promise_posts SET status = ? WHERE post_id = ?");
13         $stmt->execute([$action, $post_id]);
14         $_SESSION['toast'] = ['type' => 'success', 'message' => "Post $action successfully."];
15     }
16 }

completions.php X
Political Promise Tracker > politrack > moderator > completions.php
20 $stmt = $pdo->query("
21     SELECT cp.*, pp.title as promise_title, p.name AS politician_name, pr.proof_type, pr.file_path, u.username
22     FROM completion_posts cp
23     JOIN promise_posts pp ON cp.promise_id = pp.post_id
24     JOIN politicians p ON pp.politician_id = p.politician_id
25     JOIN proofs pr ON cp.proof_id = pr.proof_id
26     JOIN users u ON cp.user_id = u.user_id
27     WHERE cp.status = 'pending'
28     ORDER BY cp.post_date ASC
29 ");
30 $completions = $stmt->fetchAll();
```

Feature 9: Moderator Reports, Edits & User Management

- Wrote the reports query fetching all pending reports joined with the relevant content data, and a resolution handler that updates the report status and optionally removes the reported content
- Wrote the proof review query listing all proof records linked to pending submissions for moderator verification
- Wrote the edit log approval handler on approval it applies the proposed **new_value** directly to the **politicians** table and marks the log entry approved; on rejection only the log status is updated and the politicians table is left unchanged

```
reports.php X
Political Promise Tracker > politrack > moderator > reports.php
38 $stmt = $pdo->query("
39     SELECT r.*, u.username as reporter_name
40     FROM reports r
41     JOIN users u ON r.reporter_id = u.user_id
42     WHERE r.status = 'pending'
43     ORDER BY r.reported_at ASC
44 ");
```

```
proofs.php X
Political Promise Tracker > politrack > moderator > proofs.php
31 $stmt = $pdo->query("
32     SELECT p.*, u.username,
33     pp.title as promise_title, cp.completion_description
34     FROM proofs p
35     JOIN users u ON p.uploaded_by = u.user_id
36     LEFT JOIN promise_posts pp ON p.proof_id = p.proof_id
37     LEFT JOIN completion_posts cp ON cp.proof_id = p.proof_id
38     LEFT JOIN verification v ON p.proof_id = v.proof_id
39     WHERE v.verdict IS NULL OR v.verdict = 'pending'
40     ORDER BY p.uploaded_at DESC
41 ");
42 $proofs = $stmt->fetchAll();
```

```
edit.php X
Political Promise Tracker > politrack > moderator > edit.php
6 if ($SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action'])) {
11     if ($action === 'approve') {
12         $stmt = $pdo->prepare("SELECT * FROM politician_edit_log WHERE edit_id = ?");
13         $stmt->execute([$edit_id]);
14         $edit = $stmt->fetch();
15
16         if ($edit) {
17             // update politician
18             $field = $edit['field_changed'];
19             $stmt = $pdo->prepare("UPDATE politicians SET $field = ?, last_edited_by = ? WHERE politician_id = ?");
20             $stmt->execute([$_POST['new_value'], $edit['edited_by'], $edit['politician_id']]);
21
22             $pdo->prepare("UPDATE politician_edit_log SET status = 'approved' WHERE edit_id = ?")->execute([$edit_id]);
23             $_SESSION['toast'] = ['type' => 'success', 'message' => 'Edit approved and applied.'];
24         }
25     } elseif ($action === 'reject') {
26         $pdo->prepare("UPDATE politician_edit_log SET status = 'rejected' WHERE edit_id = ?")->execute([$edit_id]);
27         $_SESSION['toast'] = ['type' => 'info', 'message' => 'Edit rejected.'];
28     }
29 }
30 }
```

```
login.php X
Political Promise Tracker > politrack > login.php
10 if ($SERVER['REQUEST_METHOD'] === 'POST') {
11     if ($user && password_verify($password, $user['password'])) {
12         if ($user['is_banned']) {
13             $error = "This account has been banned.";
14         } else {
15             session_regenerate_id(true);
16             $_SESSION['user_id'] = $user['user_id'];
17             $_SESSION['username'] = $user['username'];
18             $_SESSION['role'] = $user['role'];
19
20             $_SESSION['toast'] = ['type' => 'success', 'message' => 'Welcome back, ' . htmlspecialchars($username) . '!'];
21             header("Location: index.php");
22             exit;
23         }
24     } else {
25         $error = "Invalid username or password.";
26     }
27 }
```

- Wrote the ban toggle handler that flips the `is_banned` field between 0 and 1, enforced at login where the auth system blocks any user with `is_banned = 1` from creating a session

Source Code Repository

Github Repository Link:

<https://github.com/mohammadnhiyanrahman-glitch/cse370-project-group-1>

Conclusion

The Political Promise Tracker (PolitiTrack) provides an example of how to hold political figures accountable through a single, community driven web platform. While there are still other ways to monitor political activity, the current project focuses on providing a transparent and structured system for documenting and tracking the promises made by politicians. This project is successful in achieving its main goals: registered users are able to submit promise records with proof, rate politicians, leave comments, and submit completion evidence when a promise is fulfilled; moderators are able to review and approve submissions, verify attached proofs, manage user accounts, oversee politician profile edits, and handle reports from the community. The submission of promises, the tracking of completions, and the overall accountability of politicians has been improved due to the digital and community-sourced nature of the system. Overall, the project has shown an effective way to help citizens monitor and evaluate the promises of their political representatives.

The overall goal of the project is to provide the ability to digitalize many important civic accountability functions, while at the same time allowing the general public to contribute directly to a shared record of political promises and their outcomes. Future improvements will be based on the current model, which may allow for additional functionality to be added to the system notification alerts for promised deadlines and expanded statistical reporting. We wish the team all the best as they continue their work with the Political Promise Tracker.

References

1. Bootstrap. (n.d.). *Cards — Bootstrap v5.0 components*.
<https://getbootstrap.com/docs/5.0/components/card/>
2. Hamedani, M. [Programming with Mosh]. (2019, March 19). *MySQL tutorial for beginners [full course]* [Video]. YouTube.
https://www.youtube.com/watch?v=7S_tz1z_5bA
3. Stack Overflow. (n.d.). *How can I prevent SQL injection in PHP?*
<https://stackoverflow.com/questions/60174/how-can-i-prevent-sql-injection-in-php>
4. Stack Overflow. (n.d.). *How to use PHP password_hash and password_verify?*
<https://stackoverflow.com/questions/30279321/how-to-use-php-password-hash>
5. Traversy, B. [Traversy Media]. (2022, March 30). *PHP for beginners | 3+ hour crash course* [Video]. YouTube.
<https://www.youtube.com/watch?v=BUCiSSyIGGU>

6. W3Schools. (n.d.). *PHP file upload*.
https://www.w3schools.com/php/php_file_upload.asp
7. W3Schools. (n.d.). *PHP MySQL connect*.
https://www.w3schools.com/php/php_mysql_connect.asp
8. W3Schools. (n.d.). *PHP sessions*.
https://www.w3schools.com/php/php_sessions.asp